

Guia de Rate Limiting - Implementação de Limites de Transações

Este guia explica como usar o sistema de rate limiting para aplicar limites de transações por plano.

Visão Geral

O sistema de rate limiting controla quantas transações um usuário pode realizar diariamente baseado no plano:

- **Starter (Grátis):** 50 transações/dia
- **Pro:** Ilimitado
- **Business:** Ilimitado
- **Enterprise:** Ilimitado

Como Usar em Suas Rotas API

1. Importar o Middleware

```
import { checkTransactionLimit, incrementTransactionCount } from '@lib/transaction-limiter';
import { getServerSession } from 'next-auth';
import { authOptions } from '@lib/auth';
import { NextRequest, NextResponse } from 'next/server';
```

2. Implementar em uma Rota POST

Exemplo: Rota `/api/vendas` para registrar uma venda/transação

```

export async function POST(request: NextRequest) {
  // 1. Autenticar o usuário
  const session = await getServerSession(authOptions);

  if (!session?.user?.id) {
    return NextResponse.json(
      { error: 'Não autenticado' },
      { status: 401 }
    );
  }

  // 2. Verificar o limite de transações
  const limitCheck = await checkTransactionLimit(session.user.id);

  if (!limitCheck.allowed) {
    return NextResponse.json(
      {
        error: 'Limite de transações atingido',
        message: limitCheck.message,
        tier: limitCheck.tier,
        limit: limitCheck.limit,
        remaining: limitCheck.remaining,
        // Sugerir upgrade
        suggestUpgrade: limitCheck.tier === 'starter',
      },
      { status: 429 } // HTTP 429 = Too Many Requests
    );
  }

  try {
    // 3. Processar a transação (sua lógica aqui)
    const body = await request.json();

    // Validar dados...
    // Salvar no banco...

    // 4. Após sucesso, incrementar o contador
    await incrementTransactionCount(session.user.id);

    // 5. Retornar resposta com status atualizado
    return NextResponse.json(
      {
        success: true,
        message: 'Transação registrada com sucesso',
        remaining: limitCheck.remaining - 1,
        limit: limitCheck.limit,
      },
      { status: 201 }
    );
  } catch (error) {
    console.error('Erro ao processar transação:', error);
    return NextResponse.json(
      { error: 'Erro ao processar transação' },
      { status: 500 }
    );
  }
}

```

Como Usar no Frontend

1. Mostrar o Badge de Limite

Adicione o componente `TransactionLimitBadge` em seu layout ou página:

```
// Em uma página ou componente
import { TransactionLimitBadge } from '@components/transaction-limit-badge';

export default function Dashboard() {
  return (
    <div>
      <TransactionLimitBadge /> {/* Mostra quando próximo do limite */}
      {/* Resto do conteúdo */}
    </div>
  );
}
```

2. Tratar Resposta 429 na Requisição

```
'use client';

import { useState } from 'react';
import { toast } from 'sonner';

export function VendasForm() {
  const [loading, setLoading] = useState(false);

  async function handleSubmit(e: React.FormEvent) {
    e.preventDefault();
    setLoading(true);

    try {
      const response = await fetch('/api/vendas', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          // seus dados aqui
        }),
      });

      if (response.status === 429) {
        // Limite atingido
        const data = await response.json();
        toast.error(data.message);

        // Se for plano starter, sugerir upgrade
        if (data.suggestUpgrade) {
          toast.info('Atualize seu plano para mais transações!');
          // Redirecionar para página de upgrade
        }
        return;
      }

      if (!response.ok) {
        throw new Error('Erro na requisição');
      }

      const result = await response.json();
      toast.success(result.message);

      // Mostrar transações restantes
      if (result.remaining !== undefined) {
        toast.info(`${result.remaining} transações restantes hoje`);
      }
    } catch (error) {
      toast.error('Erro ao registrar venda');
    } finally {
      setLoading(false);
    }
  }

  return (
    <form onSubmit={handleSubmit}>
      {/* Form fields */}
      <button disabled={loading}>Registrar Venda</button>
    </form>
  );
}
```



Monitorar Uso

Use a função `getUserTransactionStats` para exibir gráficos de uso:

```
import { getUserTransactionStats } from '@lib/transaction-limiter';

export async function TransactionChart({ userId }: { userId: string }) {
  const stats = await getUserTransactionStats(userId, 7); // Últimos 7 dias

  return (
    <div>
      {stats.map((day) => (
        <div key={day.date.toString()}>
          <p>{day.date.toLocaleDateString()}</p>
          <p>{day.count} / {day.limit} ({day.percentage.toFixed(0)}%)</p>
        </div>
      ))}
    </div>
  );
}
```



Resetar Contadores (Cron Job)

O sistema remove automaticamente registros com mais de 30 dias. Para limpeza manual:

```
import { resetDailyCounters } from '@lib/transaction-limiter';

// Executar periodicamente (via cron job externo ou scheduled task)
export async function cleanup() {
  const deleted = await resetDailyCounters();
  console.log(`Limpeza concluída: ${deleted} registros removidos`);
}
```



Tratamento de Erros

- **401 (Não autenticado):** Redirecionar para login
- **429 (Limite atingido):** Mostrar mensagem e sugerir upgrade para Starter
- **500 (Erro interno):** Mostrar erro genérico



Exemplos de Transações Contadas

As seguintes ações **devem** chamar `incrementTransactionCount()` após sucesso:

- ☒ Registrar venda/pedido
- ☒ Criar production plan
- ☒ Registrar movimento de estoque
- ☒ Criar relatório
- ☒ Gerar previsão de demanda

As seguintes ações **NÃO** contam como transações:

- ☒ Visualizar dados
- ☒ Buscar/filtrar ingredientes

- ❌ Gerar relatórios (apenas visualização)
- ❌ Consultar histórico

Notas de Segurança

1. **Sempre valide no servidor:** Não confie apenas em verificações do cliente
2. **Use sessions:** Sempre obtenha o `userId` da sessão autenticada
3. **Log de auditoria:** Considere registrar tentativas de exceder limite
4. **Rate limiting distribuído:** Para maior escala, considere usar Redis

Referências

- `lib/transaction-limiter.ts` - Funções principais
- `components/transaction-limit-badge.tsx` - Componente de UI
- `app/api/transaction-limit-status/route.ts` - Endpoint de status
- `lib/stripe-config.ts` - Configuração de limites por plano